Cleanup branch experiment pipeline

Each config chooses which stages run and which data sources train the U-Net.

Config

JSON recipe says what data and stages to use.

—

Fit theta

BO learns the noise recipe.

—

Generate

Make synthetic noisy image/mask pairs.

—

Build set

Stage the selected training data.

—

Train

Patchify and fine-tune the U-Net.

—

Evaluate

Compare models with metrics.

Main runner

scripts/run_experiment.py reads one experiment config, runs the requested stages in order, and writes outputs into the run folder: theta, synthetic pairs, staged training data, model, and metrics.

Big idea: the same pipeline runs different data mixtures so the experiments are comparable.

Experiment 1: clean + synthetic

Can BO-generated synthetic noise help without training directly on real noisy images?

Uses

Clean image/mask pairs

Reference noisy images only to fit theta

Synthetic noisy pairs for training

Skips or limits

Real non-clean pairs are not used directly in U-Net training.

What happens

1. noise_fit.py fits theta from clean + reference noisy images.

2. generate_synthetic.py creates synthetic noisy versions of the clean pairs.

3. train_experiment.py trains on clean + synthetic pairs.

4. evaluate_experiment.py scores the trained model.

Output to compare: model.keras + metrics.json.

Experiment 2: clean + real noisy

How much does direct real non-clean training data help by itself?

Uses

Clean image/mask pairs

Real non-clean image/mask pairs

No synthetic images

Skips or limits

No BO fitting.

No synthetic generation.

What happens

1. build_training_set.py stages clean + real non-clean pairs.

2. train_experiment.py trains the U-Net on that staged set.

3. evaluate_experiment.py scores the trained model.

This is the simplest direct-data baseline.

Output to compare: model.keras + metrics.json.

Experiment 3: clean + synthetic + real noisy

Does the best performance come from combining fake noisy data and real noisy data?

Uses

Clean image/mask pairs

Synthetic BO-noised pairs

Real non-clean image/mask pairs

Skips or limits

Nothing major; this is the combined-data experiment.

What happens

1. noise_fit.py learns theta.

2. generate_synthetic.py creates synthetic noisy pairs.

3. build_training_set.py stages clean + synthetic + real non-clean pairs.

4. training.py fine-tunes the U-Net, then evaluation.py scores it.

Output to compare: model.keras + metrics.json.

Experiment 4: r / 3-r split

When real noisy images are scarce, should they fit theta, train the U-Net, or both?

Uses

r real non-clean images to fit theta

3-r real non-clean images for training

Clean + synthetic pairs

Skips or limits

Avoids using the exact same real images for both theta fitting and direct U-Net training.

What happens

1. Split real non-clean images into two folders.

2. Fit theta using only the r-image folder.

3. Generate synthetic noisy pairs from that theta.

4. Train on clean + synthetic + the remaining 3-r real pairs.

This is the most careful comparison.

Output to compare: model.keras + metrics.json.

Script cheat sheet

The scripts are thin command-line wrappers around the reusable experiment modules.

fit_noise.py

Fits BO theta from clean + reference noisy images.

train_experiment.py

Builds the training set, patchifies it, and fine-tunes the U-Net.

generate_synthetic.py

Uses theta to make synthetic noisy image/mask pairs.

evaluate_experiment.py

Loads a model and scores it on held-out image/mask pairs.

build_training_set.py

Copies selected clean/synthetic/real pairs into one staged folder.

run_experiment.py

Reads a JSON config and runs the whole recipe end-to-end.

How to run one

```
conda run -n segmenteverygrain python scripts/run_experiment.py --config
experiments/configs/exp1_clean_synthetic.json
```

Use the segmenteverygrain conda environment for scipy/tensorflow.